

Instituto Tecnológico y de Estudios Superiores de Monterrey
Campus Monterrey



TE3001B

Fundamentación de robótica

Mini-reto: Crear un controlador PID para un motor simulado

Alumnos:

Josue Ureña Valencia	IRS A01738940
César Arellano Arellano	IRS A00839373
Jose Eduardo Sanchez Martinez	IRS A01738476
Rafael André Gamiz Salazar	IRS A00838280

Fecha de entrega:

26 de febrero del 2026

Índice

Introducción.....	3
Fase 1. Controlador PID y modificación de parámetros.....	4
Arquitectura del sistema.....	4
Nodo controller.....	4
Launch File.....	9
Controlador PID sin tuneear.....	10
Controlador PID tuneado.....	11
Fase 2. Generar diferentes señales.....	12
Square.....	14
Triangle.....	15
Step.....	16
Sine.....	17
Fase 3. Grupos independientes.....	18
Launch file.....	18
Archivo de configuración.....	19
Comportamiento del setpoint de los 3 grupos.....	21

Video:  Canva

Introducción

El presente documento describe el paso a paso de la implementación técnica del Mini Challenge asignado a la semana 2 del curso. El objetivo es diseñar e implementar un controlador PID para regular la velocidad de un motor DC simulado por el nodo `/motor_sys`, ya asignado por MCR2. Debe desarrollarse sin el uso de librerías externas de control y utilizando únicamente NumPy para operaciones matemáticas.

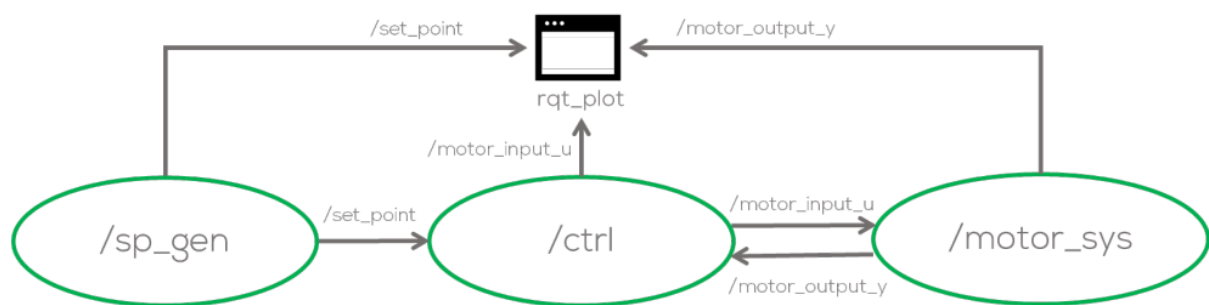
El reto se divide en tres fases: la primera consiste en la implementación del controlador PID con parámetros ajustables mientras se encuentra en ejecución; la segunda debe ser capaz de generar múltiples tipos de señal en runtime; y en la tercera el sistema debe operar tres grupos de control completamente independientes mediante namespaces de ROS2.

Fase 1. Controlador PID y modificación de parámetros

En esta fase se implementa el nodo controlador PID, el launch file que levanta los tres nodos del sistema, y la capacidad de modificar las ganancias del controlador desde la terminal en tiempo de ejecución sin reiniciar ningún nodo.

Arquitectura del sistema

El sistema de control de lazo cerrado está compuesto por tres nodos que se comunican a través de tópicos ROS2. El flujo de información es el siguiente:



Nodo controller

Su función es leer el set point y la salida del motor, calcular la acción de control y publicarla hacia el motor.

Declaración de parámetros y variables de estado

```
Python
class PIDController(Node):

    def __init__(self):
        super().__init__("controller")

        # Parameters
        self.declare_parameter("kp", 1.0)
        self.declare_parameter("ki", 0.0)
        self.declare_parameter("kd", 0.0)
        self.declare_parameter("Ts", 0.02)

        self.kp = self.get_parameter("kp").value
        self.ki = self.get_parameter("ki").value
        self.kd = self.get_parameter("kd").value
        self.Ts = self.get_parameter("Ts").value
```

```

# States
self.e_prev = 0.0
self.integral = 0.0
self.y = 0.0
self.sp = 0.0

```

Los parámetros `kp`, `ki`, `kd` y `Ts` se declaran con valores por defecto. Al inicio se leen del servidor de parámetros, que puede ser sobrescrito desde el launch file o un archivo YAML. Las variables de estado (`e_prev`, `integral`) se inicializan en cero.

Suscriptores, publicador, timer y parameter callback

Python

```

# Subscribers
self.create_subscription(Float32, "set_point", self.sp_callback, 10)
self.create_subscription(Float32, "motor_speed_y", self.y_callback, 10)

# Publisher
self.pub = self.create_publisher(Float32, "motor_input_u", 10)

# Timer
self.timer = self.create_timer(self.Ts, self.control_loop)

#Parameter Callback
self.add_on_set_parameters_callback(self.parameters_callback)

```

El nodo se suscribe a dos tópicos: `set_point` para recibir la referencia y `motor_speed_y` para recibir la velocidad medida del motor. Publica la señal de control en `motor_input_u`. El timer dispara el loop de control cada `Ts` segundos.

Loop de control PID discreto

Python

```

def control_loop(self):
    e = self.sp - self.y

    self.integral += e * self.Ts
    derivative = (e - self.e_prev) / self.Ts

    u = (
        self.kp * e
        + self.ki * self.integral

```

```

        + self.kd * derivative
    )
    msg = Float32()
    msg.data = float(u)
    self.pub.publish(msg)

    self.e_prev = e

```

Se implementa la ley de control PID en tiempo discreto. El error se calcula como la diferencia entre el set point y la salida del motor. La integral acumula el error multiplicado por el periodo de muestreo. La derivada aproxima la tasa de cambio del error usando la diferencia hacia atrás:

$$u[k] = K_p \cdot e[k] + K_i \cdot I[k] + K_d \cdot (e[k] - e[k - 1]) / T_s$$

Validación de parámetros en tiempo de ejecución

Python

```

def parameters_callback(self, params):
    for param in params:
        if param.name == "kp":
            if param.value < 0.0:
                self.get_logger().warn("Kp cannot be negative.")
                return SetParametersResult(successful=False, reason="Kp
must be >= 0")
            else:
                self.kp = param.value # Update internal variable
                self.get_logger().info(f"Kp updated to {self.kp}")

        if param.name == "ki":
            if param.value < 0.0:
                self.get_logger().warn("Ki cannot be negative.")
                return SetParametersResult(successful=False, reason="Ki
must be >= 0")
            else:
                self.ki = param.value # Update internal variable
                self.get_logger().info(f"Ki updated to {self.ki}")

        if param.name == "kd":
            if param.value < 0.0:
                self.get_logger().warn("Kd cannot be negative.")
                return SetParametersResult(successful=False, reason="Kd
must be >= 0")

```

```

        else:
            self.kd = param.value # Update internal variable
            self.get_logger().info(f"Kd updated to {self.kd}")

    if param.name == "Ts":
        if param.value < 0.0:
            self.get_logger().warn("Ts cannot be negative.")
            return SetParametersResult(successful=False, reason="Ts
must be >= 0")
        else:
            self.Ts
            = param.value # Update internal variable
            self.get_logger().info(f"Ts updated to {self.Ts}")

    return SetParametersResult(successful=True)

```

El callback `parameters_callback` se ejecuta automáticamente cada vez que se modifica un parámetro en runtime. Si el valor es negativo, el cambio es rechazado y se notifica al usuario mediante un mensaje. Si es válido, la variable interna se actualiza al instante sin necesidad de reiniciar el nodo.

La modificación de parámetros desde terminal se realiza con el comando:

```

Shell
ros2 param set /control kp 3.5

```

lo que dispara el callback inmediatamente sin interrumpir el loop de control.

Podemos comprobar la topología de los nodos y tópicos de esta primera fase con el `qt_graph`, lo que se ve es el lazo cerrado que se implementó:



- `/sp_gen` publica en `/set_point`
- `/control` recibe el set point, calcula `u` y publica en `/motor_input_u`
- `/motor_sys` recibe la señal de control y publica de vuelta en `/motor_speed_y`

Si esto no fuera suficiente, el comando:

```
Shell  
ros2 node list
```

nos devuelve la lista de todos los nodos que están activos en ese momento en el sistema ROS2

```
ed@je:~/ros2_challenge_mcr2/ros2_challenge1$ ros2 node list  
/control  
/motor_sys  
/sp_gen  
ed@je:~/ros2_challenge_mcr2/ros2_challenge1$
```

Así mismo, el comando:

```
Shell  
ros2 param list
```

nos devuelve todos los parámetros declarados por cada nodo activo en el sistema ROS2

```
ed@je:~/ros2_challenge_mcr2/ros2_challenge1$ ros2 param list  
/control:  
  Ts  
  kd  
  ki  
  kp  
  use_sim_time  
/motor_sys:  
  initial_conditions  
  sample_time  
  sys_gain_K  
  sys_tau_T  
  use_sim_time  
/sp_gen:  
  signal_type  
  use_sim_time  
ed@je:~/ros2_challenge_mcr2/ros2_challenge1$
```


Launch File

El launch file levanta los tres nodos del sistema en un solo comando. Los parámetros de cada nodo se pasan directamente como diccionario. El controlador inicia con $K_p=2.0$, $K_i=0$ y $K_d = 0$ como punto de partida:

```
Python
from launch import LaunchDescription
from launch_ros.actions import Node

def generate_launch_description():
    motor_node = Node(name="motor_sys",
                      package='motor_control',
                      executable='dc_motor',
                      emulate_tty=True,
                      output='screen',
                      parameters=[{
                          'sample_time': 0.01,
                          'sys_gain_K': 2.16,
                          'sys_tau_T': 0.05,
                          'initial_conditions': 0.0,
                      }
                      ])

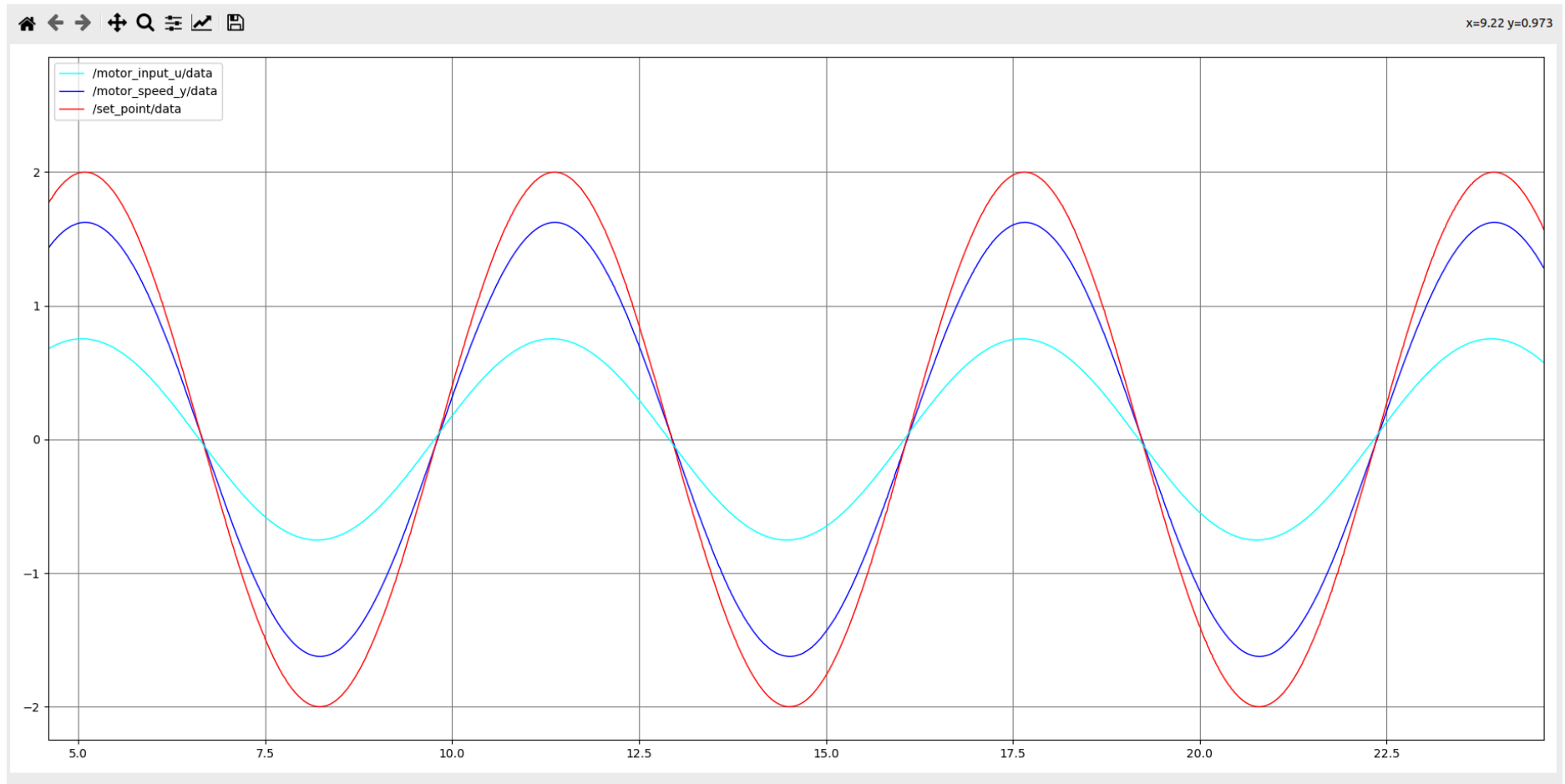
    control_node = Node(name="control",
                       package='motor_control',
                       executable='controller',
                       emulate_tty=True,
                       output='screen',
                       parameters=[{
                           'kp': 2.0,
                           'ki': 0.0,
                           'kd': 0.0,
                           'Ts': 0.01,
                       }
                       ])

    sp_node = Node(name="sp_gen",
                   package='motor_control',
                   executable='set_point',
                   emulate_tty=True,
                   output='screen',
                   parameters=[{
                       'signal_type': 'sine',
                   }
                   ])

    l_d = LaunchDescription([motor_node, control_node, sp_node])
    return l_d
```

Controlador PID sin tunear

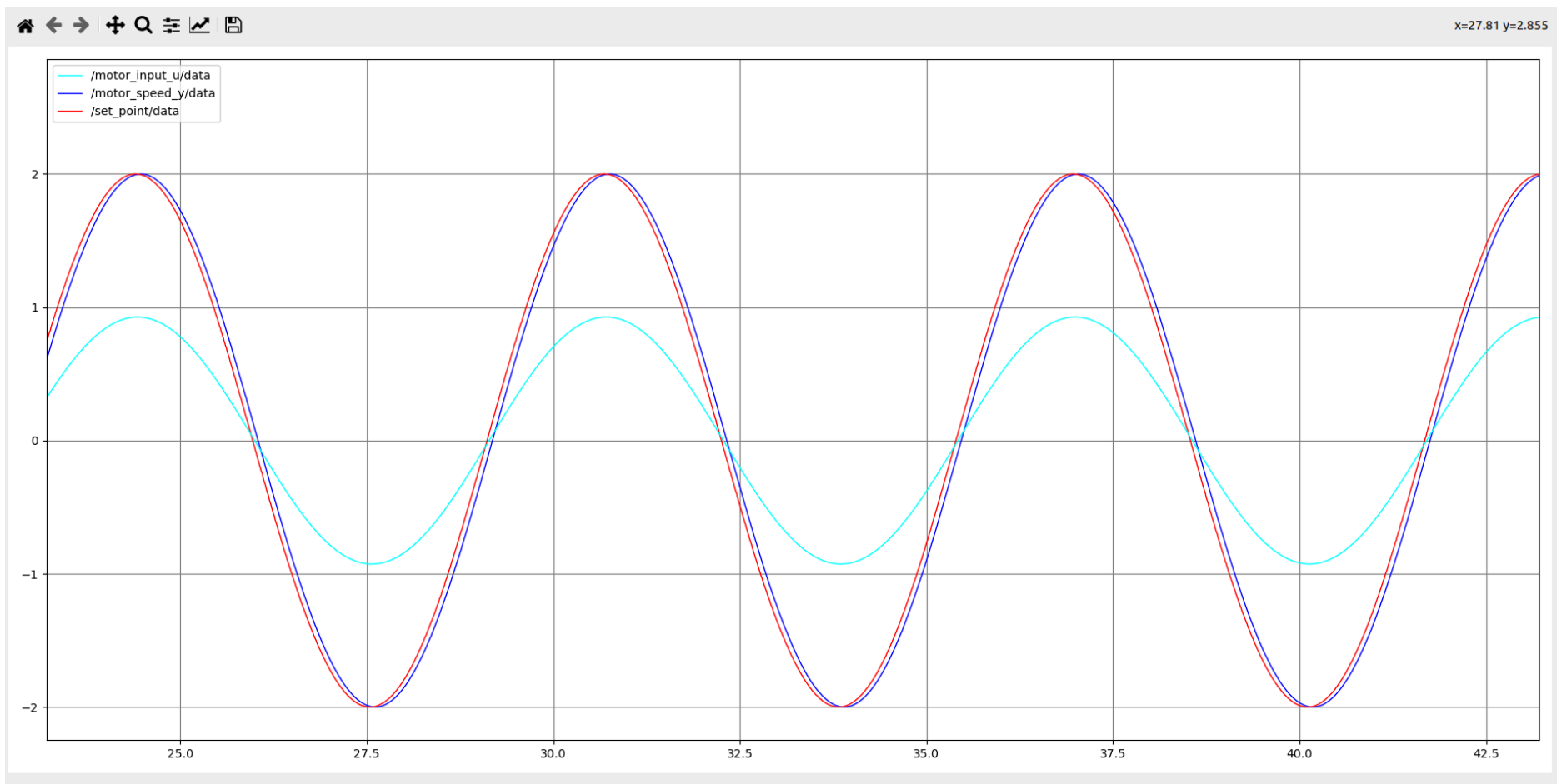
$K_p=2.0$, $K_i=0$ y $K_d=0$



Controlador PID tuneado

```
ed@je:~/ros2_challenge_mcr2/ros2_challenge1$ ros2 param set /control kp 0.1
Set parameter successful
ed@je:~/ros2_challenge_mcr2/ros2_challenge1$ ros2 param set /control ki 7.0
Set parameter successful
```

$K_p=0.1$, $K_i=7.0$ y $K_d = 0$



Fase 2. Generar diferentes señales

En esta fase se extiende el nodo generador de set point para que genere múltiples tipos de señal de referencia, seleccionables en tiempo de ejecución mediante el parámetro `signal_type` sin detener el sistema.

set_point.py

```
Python
#Class Definition
class SetPointPublisher(Node):
    def __init__(self):
        super().__init__('set_point_node')

        self.declare_parameter('signal_type', 'sine')
        self.signal_type = self.get_parameter('signal_type').value

        # Retrieve sine wave parameters
        self.amplitude = 2.0
        self.omega = 1.0
        self.timer_period = 0.01 #seconds

        #Create a publisher and timer for the signal
        self.signal_publisher = self.create_publisher(Float32, 'set_point',
10) ## CHECK FOR THE NAME OF THE TOPIC
        self.timer = self.create_timer(self.timer_period, self.timer_cb)

        #Create a messages and variables to be used
        self.signal_msg = Float32()
        self.start_time = self.get_clock().now()
        #Parameter Callback
        self.add_on_set_parameters_callback(self.parameters_callback)
```

El parámetro `signal_type` controla el tipo de señal generada. Se declara al inicializar el nodo y puede modificarse en runtime. El timer ejecuta el callback `timer_cb` cada 0.01 s, que calcula la señal correspondiente basándose en el tiempo transcurrido desde el arranque.

Tipos de señal implementados

```
Python
def timer_cb(self):
    #Calculate elapsed time
```

```

        elapsed_time = (self.get_clock().now() -
self.start_time).nanoseconds/1e9
        if self.signal_type == 'sine':
            signal = self.amplitude * np.sin(self.omega * elapsed_time)

        elif self.signal_type == 'square':
            signal = self.amplitude * np.sign(np.sin(self.omega *
elapsed_time))

        elif self.signal_type == 'triangle':
            signal = (2 * self.amplitude / np.pi) *
np.arcsin(np.sin(self.omega * elapsed_time))

        elif self.signal_type == 'step':
            if elapsed_time >= 2.0:
                signal = self.amplitude
            else:
                signal = 0.0

```

El método `timer_cb` evalúa el valor de `signal_type` en cada ciclo y genera la señal correspondiente:

- `sine`: señal sinusoidal
- `square`: señal cuadrada
- `triangle`: señal triangular
- `step`: escalón unitario

El cambio de señal en runtime se realiza con el comando:

```

Shell
ros2 param set /sp_gen signal_type square

```

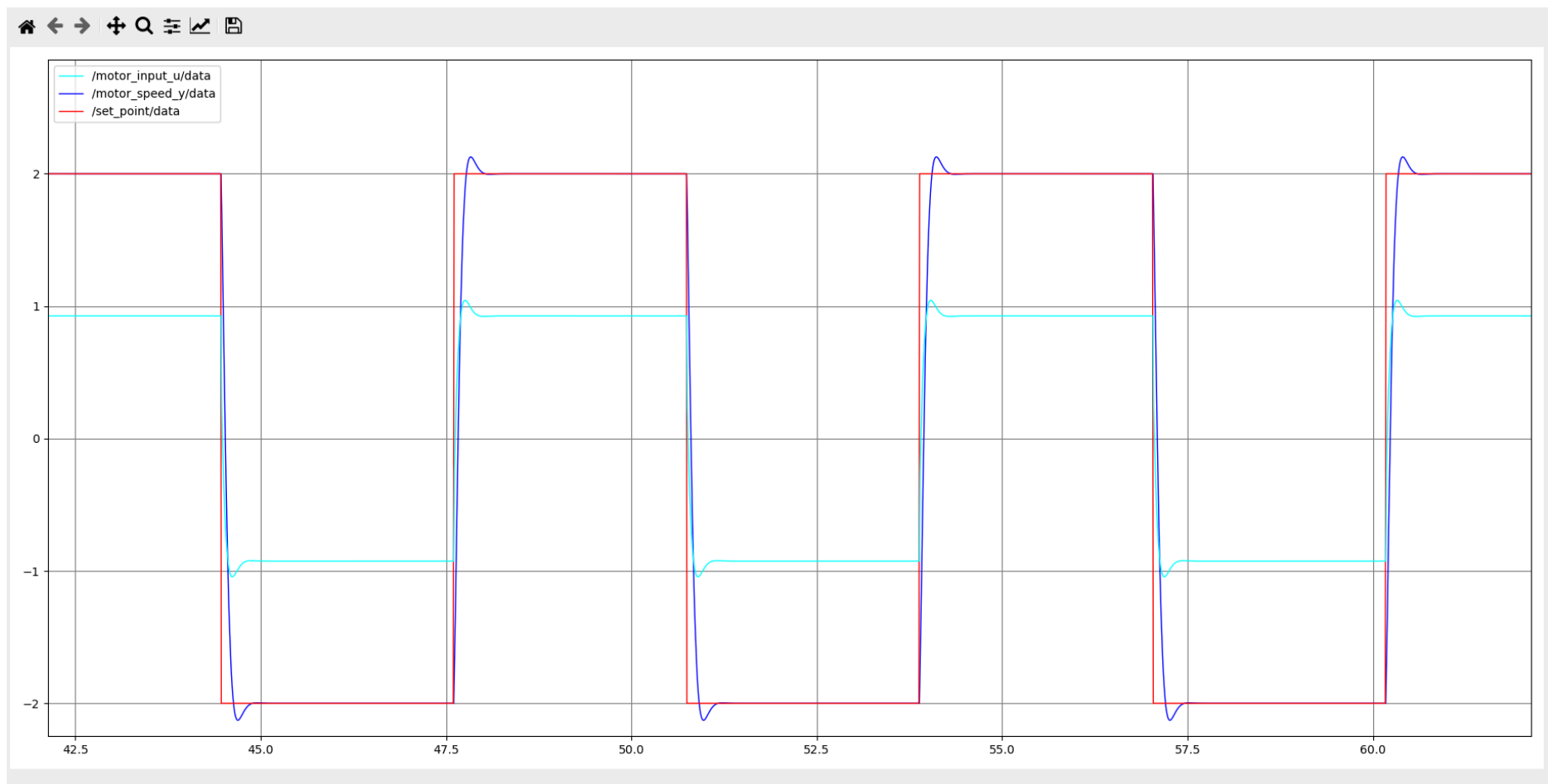
```

ed@je:~/ros2_challenge_mcr2/ros2_challenge1$ ros2 param set /sp_gen signal_type square
Set parameter successful
ed@je:~/ros2_challenge_mcr2/ros2_challenge1$ ros2 param set /sp_gen signal_type triangle
Set parameter successful
ed@je:~/ros2_challenge_mcr2/ros2_challenge1$ ros2 param set /sp_gen signal_type step
Set parameter successful
ed@je:~/ros2_challenge_mcr2/ros2_challenge1$ ros2 param set /sp_gen signal_type sine
Set parameter successful
ed@je:~/ros2_challenge_mcr2/ros2_challenge1$ ^C

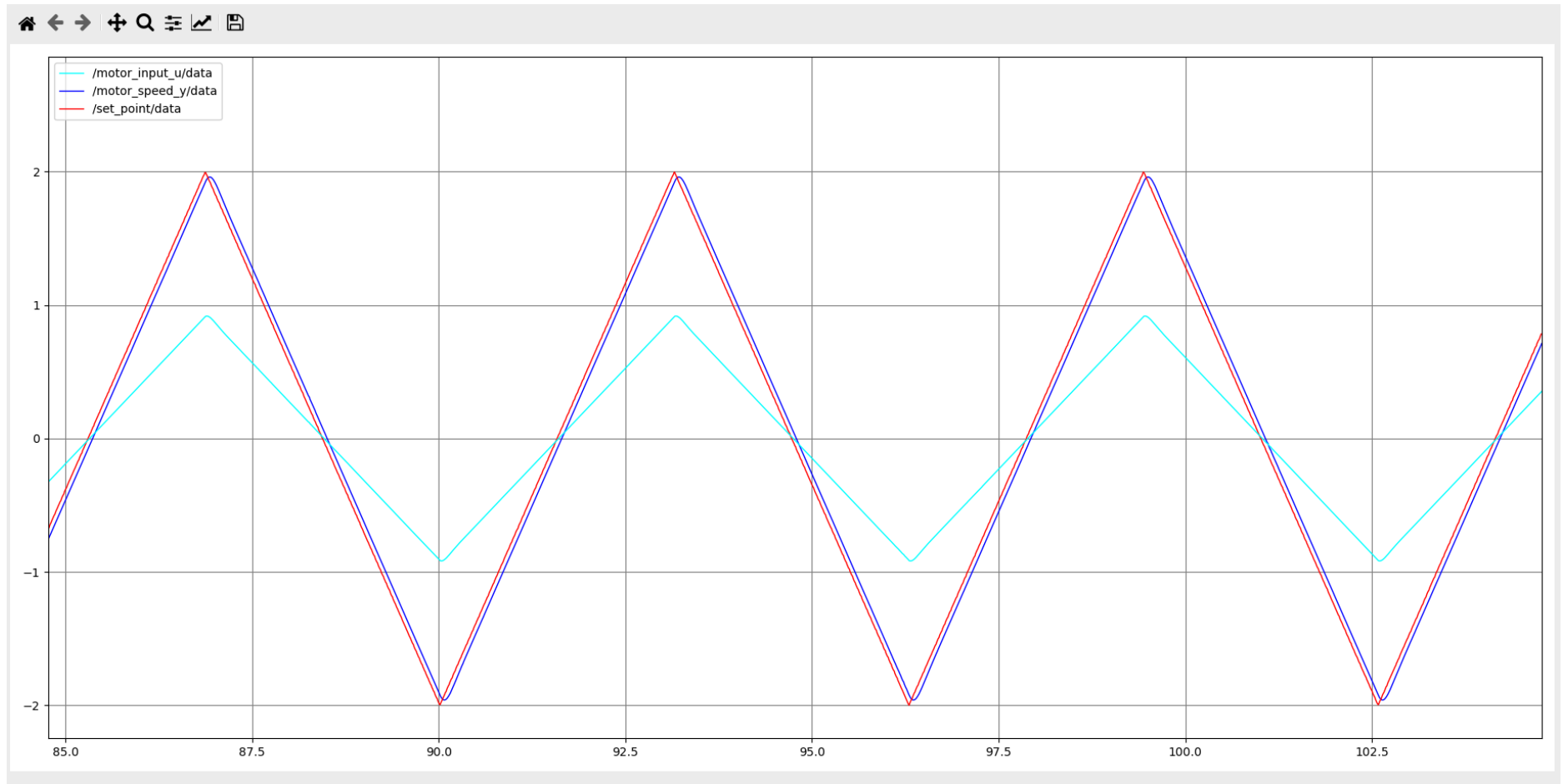
```

A continuación se muestran las 4 diferentes señales que se pueden generar:

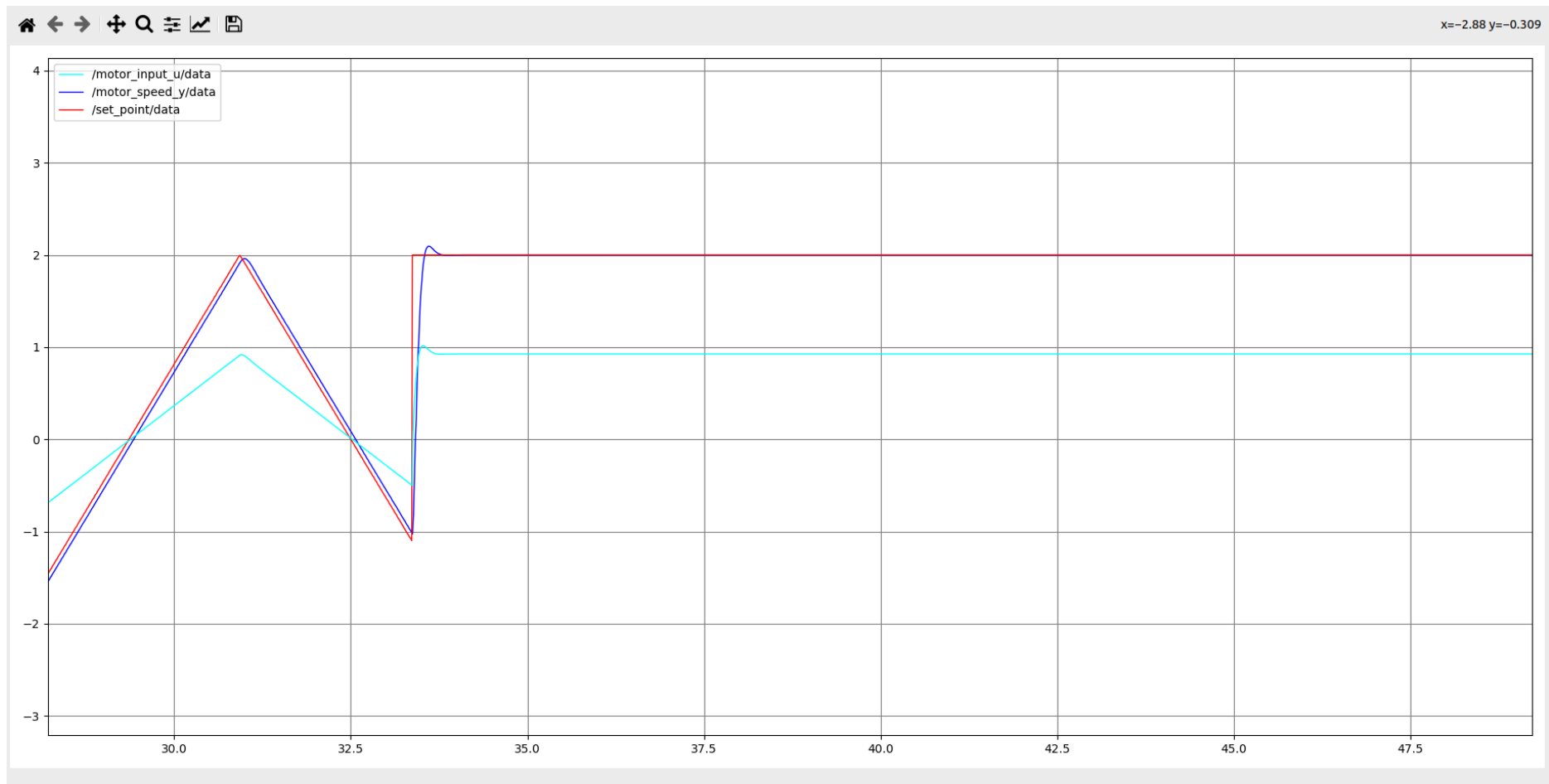
Square



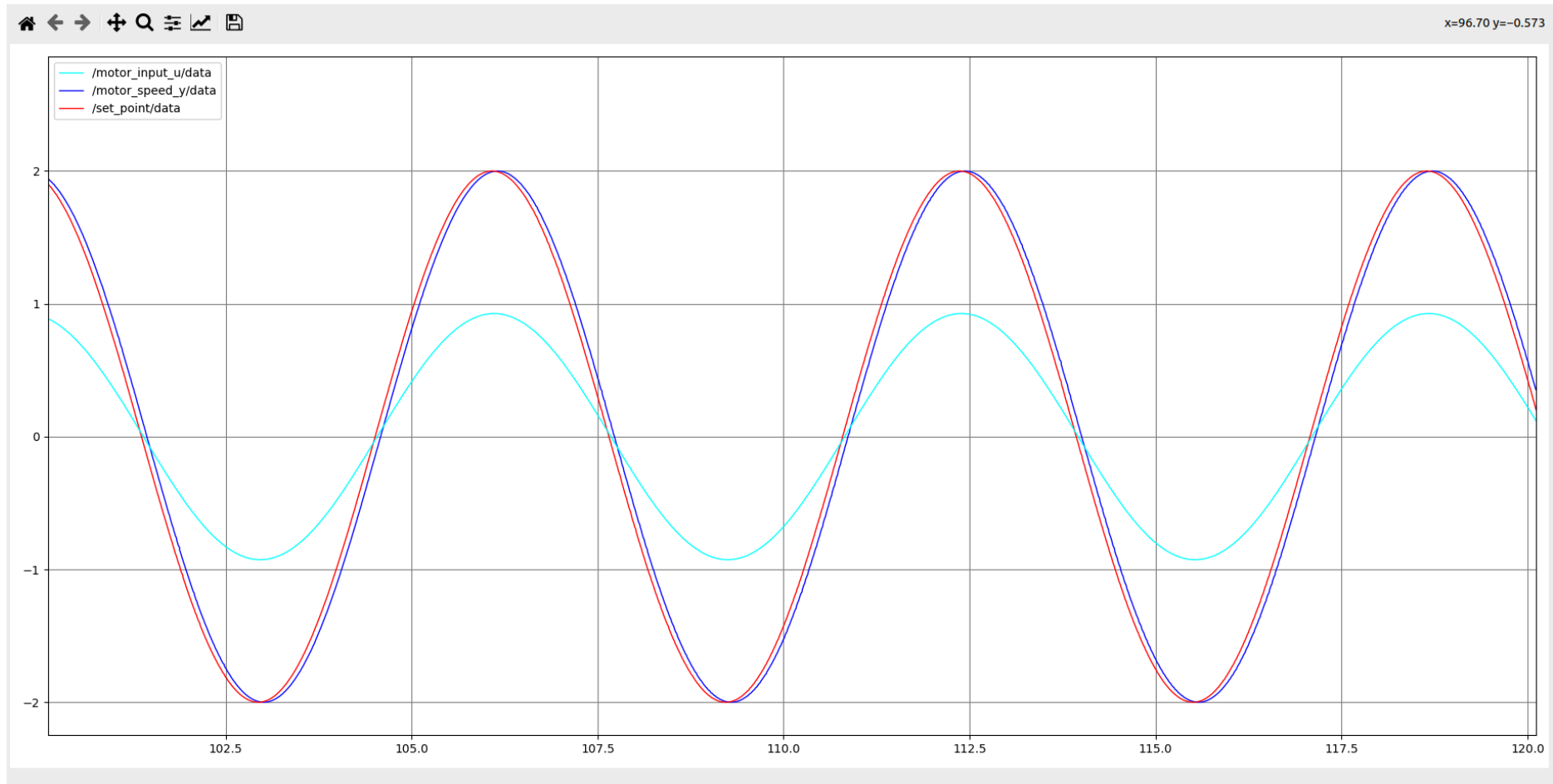
Triangle



Step



Sine



Fase 3. Grupos independientes

En esta fase se escala el sistema para correr tres instancias del esquema de control en paralelo, completamente aisladas entre sí mediante namespaces de ROS2: group1, group2 y group3. Cada grupo tiene su propio motor, controlador y generador de set point, sin interferencia entre tópicos.

Launch file

Adaptamos a un archivo YAML de configuración centralizado, cargado mediante `get_package_share_directory`.

```
Python
def generate_launch_description():

    config = os.path.join(
        get_package_share_directory('motor_control'),
        'config',
        'params.yaml'
    )

    motor_node_1 = Node(name="motor_sys_1",
        package='motor_control',
        executable='dc_motor',
        emulate_tty=True,
        output='screen',
        namespace="group1",
        parameters=[config]
    )

    sp_node_1 = Node(name="sp_gen_1",
        package='motor_control',
        executable='set_point',
        emulate_tty=True,
        output='screen',
        namespace="group1",
        parameters=[config]
    )

    control_node_1 = Node(name="control_1",
        package='motor_control',
        executable='controller',
        emulate_tty=True,
        output='screen',
```

```

        namespace="group1",
        parameters=[config]
    )

    // Se sigue el mismo patron para grupo 2 y 3

    l_d = LaunchDescription([
        motor_node_1, sp_node_1, control_node_1,
        motor_node_2, sp_node_2, control_node_2,
        motor_node_3, sp_node_3, control_node_3])

    return l_d

```

Cada nodo recibe el namespace correspondiente, lo que hace que todos sus tópicos queden aislados bajo ese único id.

Archivo de configuración

El archivo YAML especifica los parámetros de cada nodo usando su ruta completa (namespace + nombre), lo que permite asignar valores distintos a cada grupo. Cada grupo opera con parámetros de motor y señal de referencia diferentes, permitiendo comparar el comportamiento del controlador ante distintas plantas y referencias:

```

Python
/group1/motor_sys_1:
  ros__parameters:
    sys_gain_K: 4.0
    sys_tau_T: 0.9
    sample_time: 0.01
    initial_conditions: 0.0

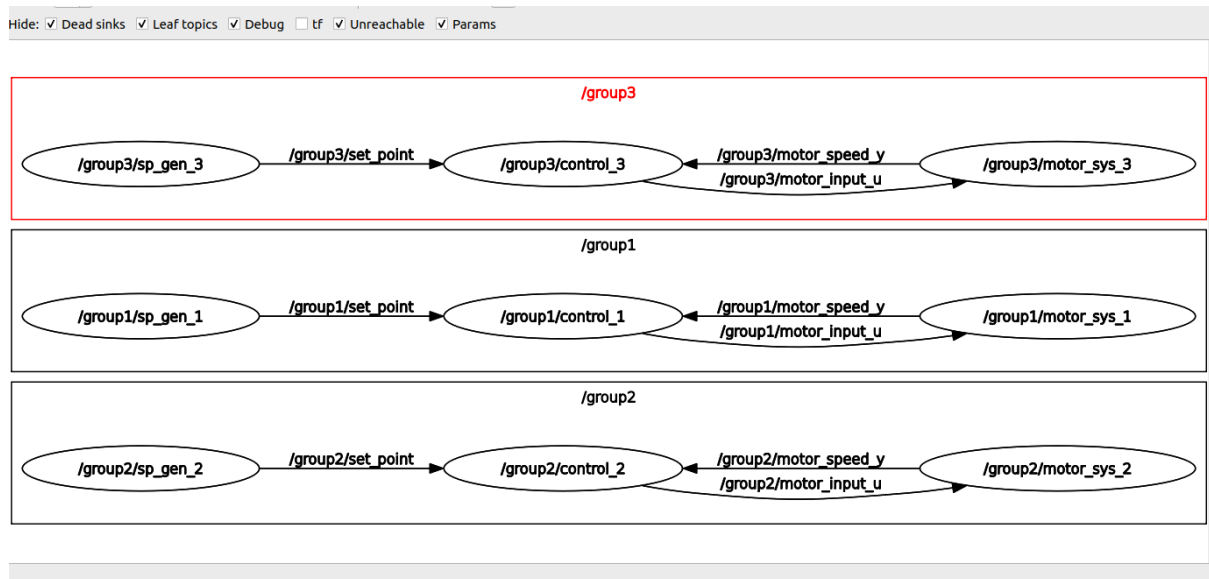
/group1/control_1:
  ros__parameters:
    kp: 0.1
    ki: 7.0
    kd: 0.0
    Ts: 0.01

/group1/sp_gen_1:
  ros__parameters:
    signal_type : "sine"

// Se sigue el mismo patron para grupo 2 y 3

```

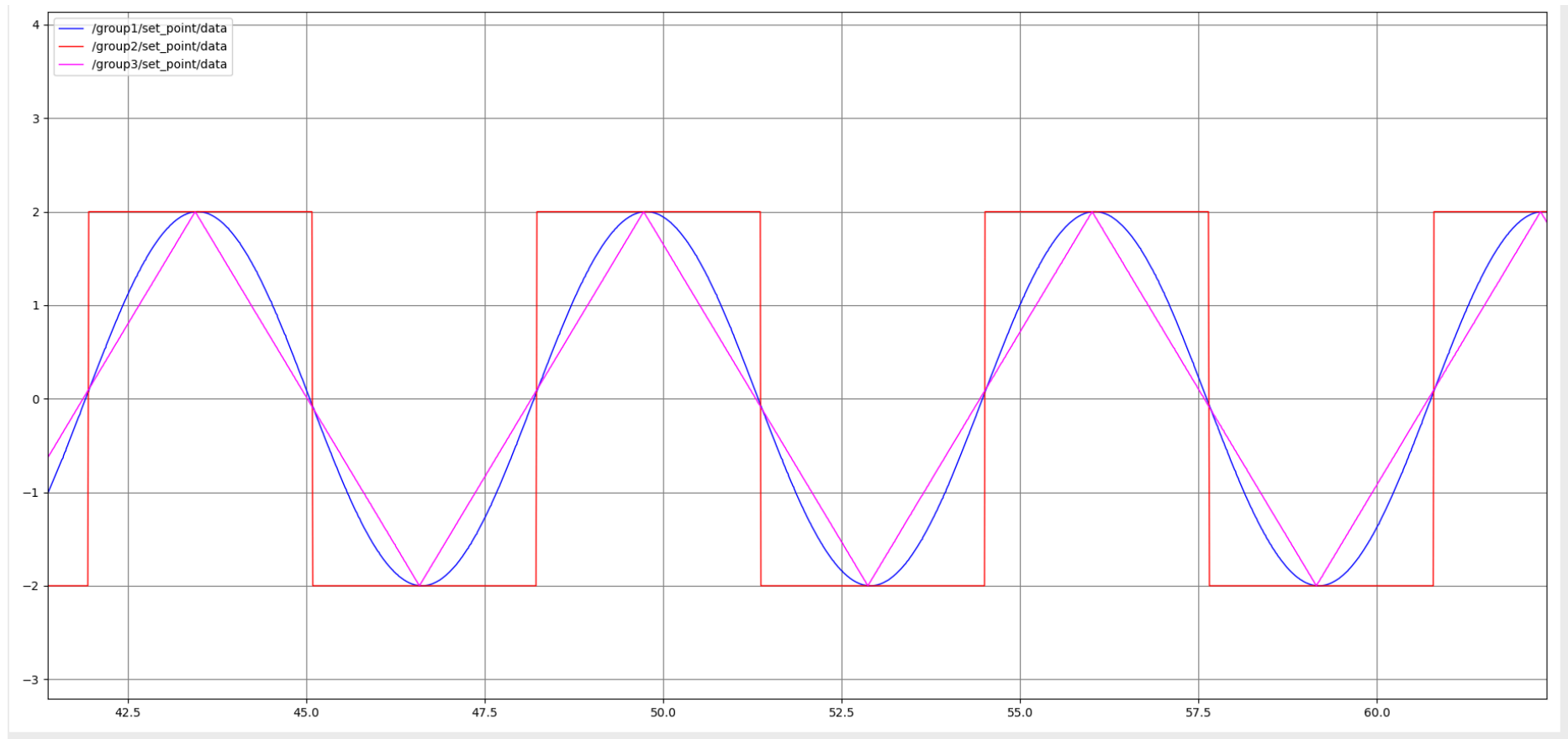
La verificación de aislamiento se realiza con `rqt_graph`, donde se observan los tres subgrafos de nodos completamente separados, y con `ros2 topic list` para confirmar que los tópicos de cada grupo no se cruzan entre sí.



Y usando `rqt_plot` podemos ver cómo cada grupo está trabajando de manera independiente. Para observar esto, el archivo YAML se le asignó una señal de setpoint diferente para cada grupo:

- grupo1 : señal sinusoidal
- grupo2 : señal cuadrada
- grupo3: señal triangular

Comportamiento del setpoint de los 3 grupos



Y como se puede observar, se comprueba que cada grupo tiene una señal de setpoint diferente, comprobando que trabajan de manera independiente.